Laboratory Activity #02

Distributed Systems Programming



Why gRPC?



- gRPC (https://grpc.io/) is a modern open-source **high performance** framework implementing the remote procedure call (***RPC***) paradigm.
- The main features of gRPC are:
  1) simple service definition (Protocol Buffer);
  2) high performance and scalability;
  3) bi-directional streaming support;
  4) multi-language and multi-platform.

Why gRPC?



- gRPC (https://grpc.io/) is a modern open-source **high performance** framework implementing the remote procedure call (***RPC***) paradigm.
- The main features of gRPC are:
  1) simple service definition (Protocol Buffer);
  2) high performance and scalability;
  3) bi-directional streaming support;
  4) multi-language and multi-platform.

gRPC is suitable for **Machine-To-Machine (M2M)** communications.

Topics of the Laboratory Session



Laboratory Session #02 covers the following activities:

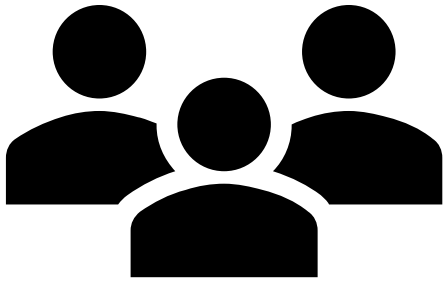


Definition of new **REST APIs** exposed by the Film Manager service for the management of **images**

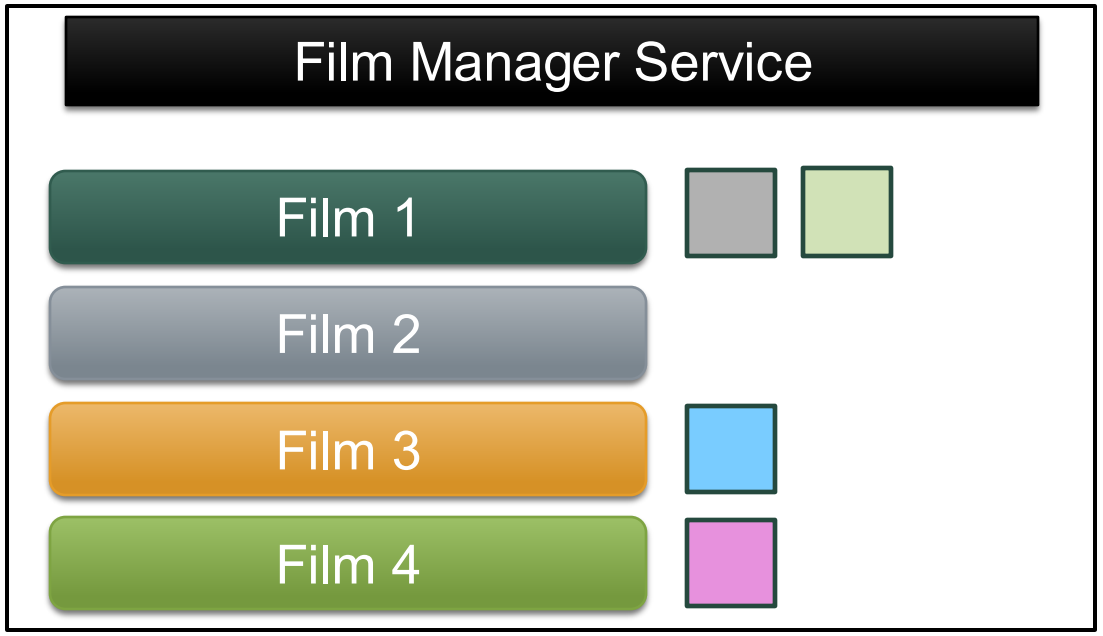


Integration of a **gRPC client** functionality in the implementation of the Film Manager service

Image Management in the Film Manager service (I)

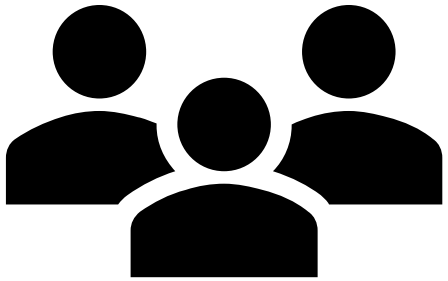






- A user can associate **multiple** images to a film (if she is the owner).
- The allowed **media types** are: PNG, JPEG, GIF.
- The service stores the image with its media type.

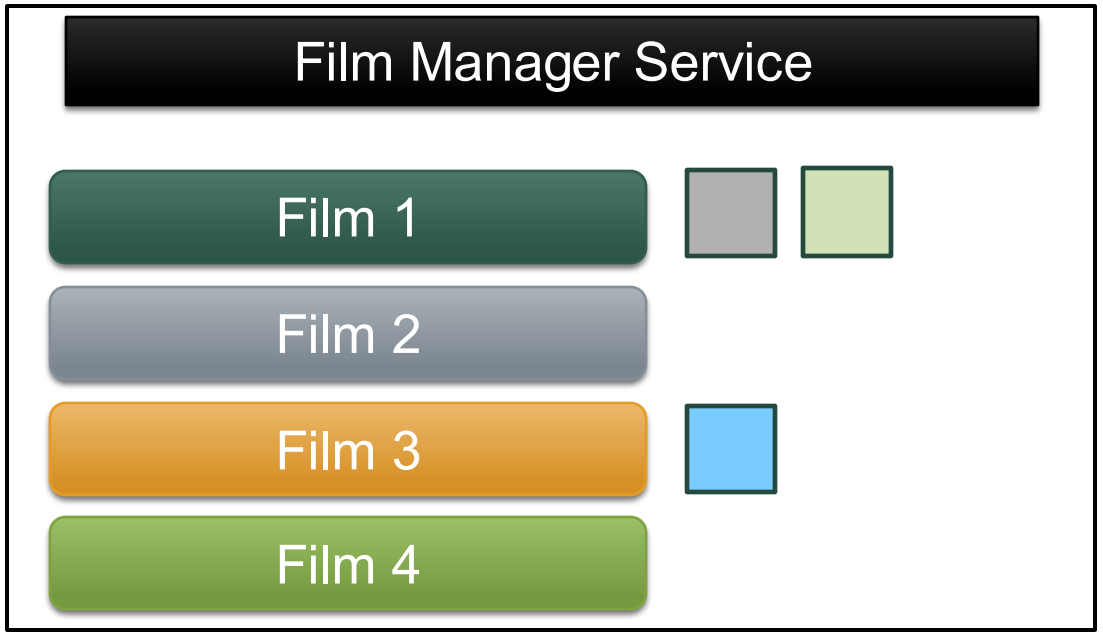
Image Management in the Film Manager service (I)



POST

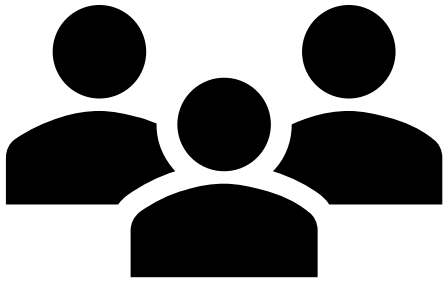
for Film 4





- A user can associate **multiple** images to a film (if she is the owner).
- The allowed **media types** are: PNG, JPEG, GIF.
- The service stores the image with its media type.

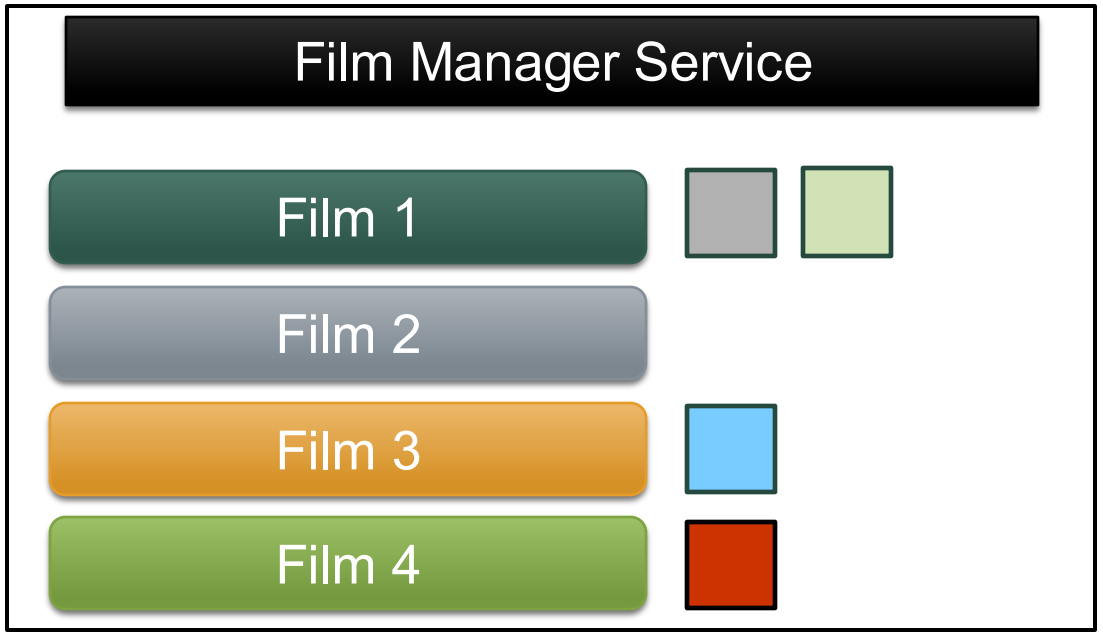
Image Management in the Film Manager service (I)



POST

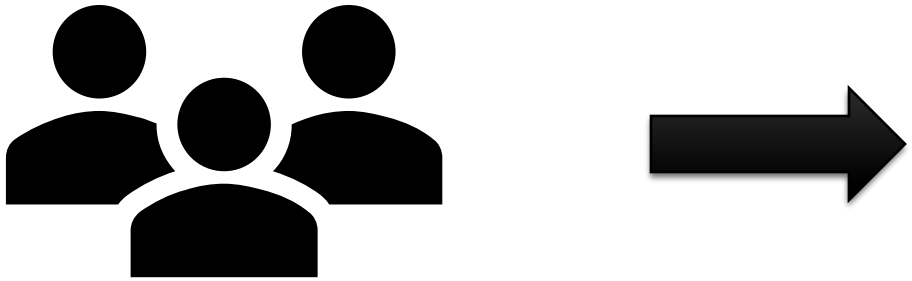
for Film 4

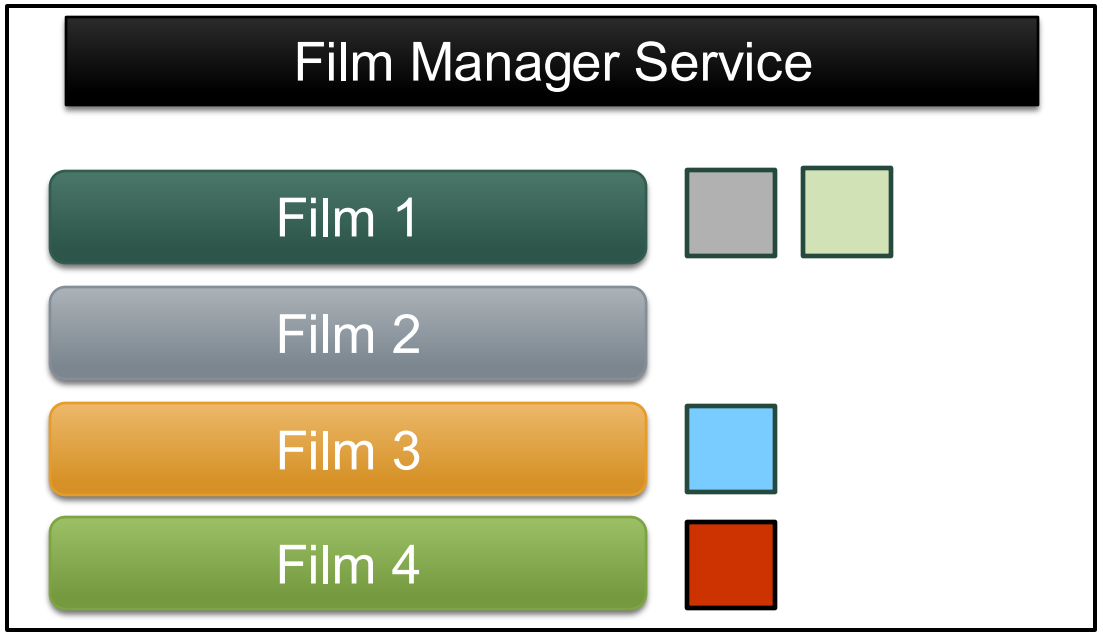




- A user can associate **multiple** images to a film (if she is the owner).
- The allowed **media types** are: PNG, JPEG, GIF.
- The service stores the image with its media type.

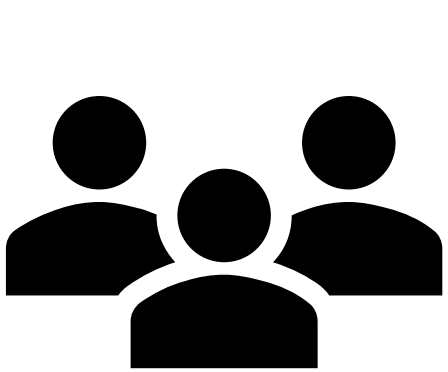
Image Management in the Film Manager service (II)



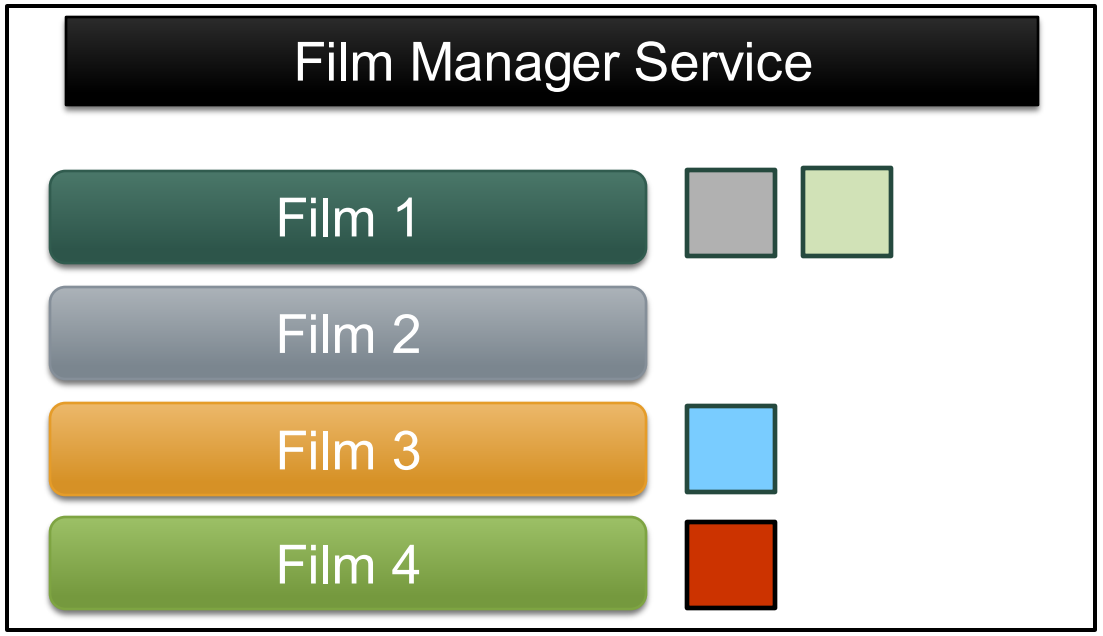


- A user can **delete** an image associated to a film (if she is the owner).
- The image is not saved anymore server-side.

Image Management in the Film Manager service (II)

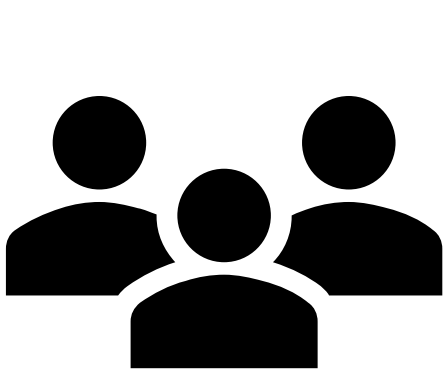




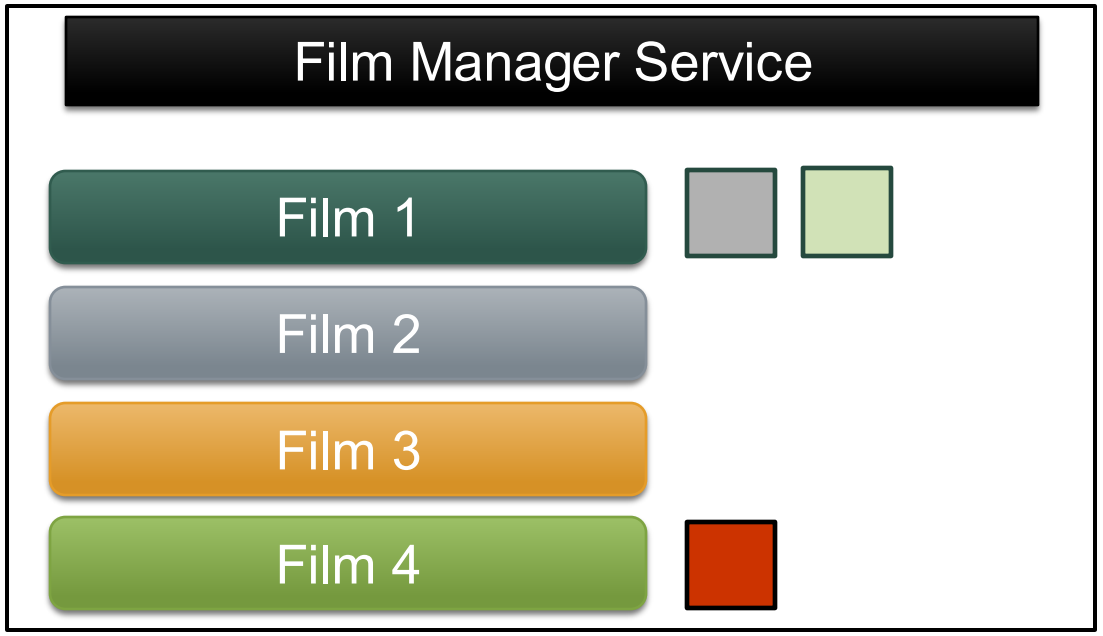


- A user can **delete** an image associated to a film (if she is the owner).
- The image is not saved anymore server-side.

Image Management in the Film Manager service (II)

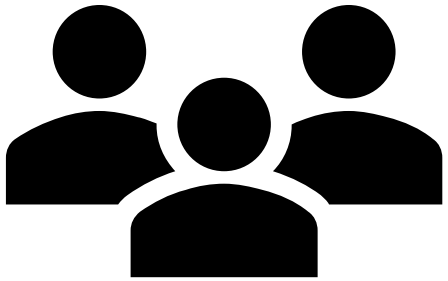




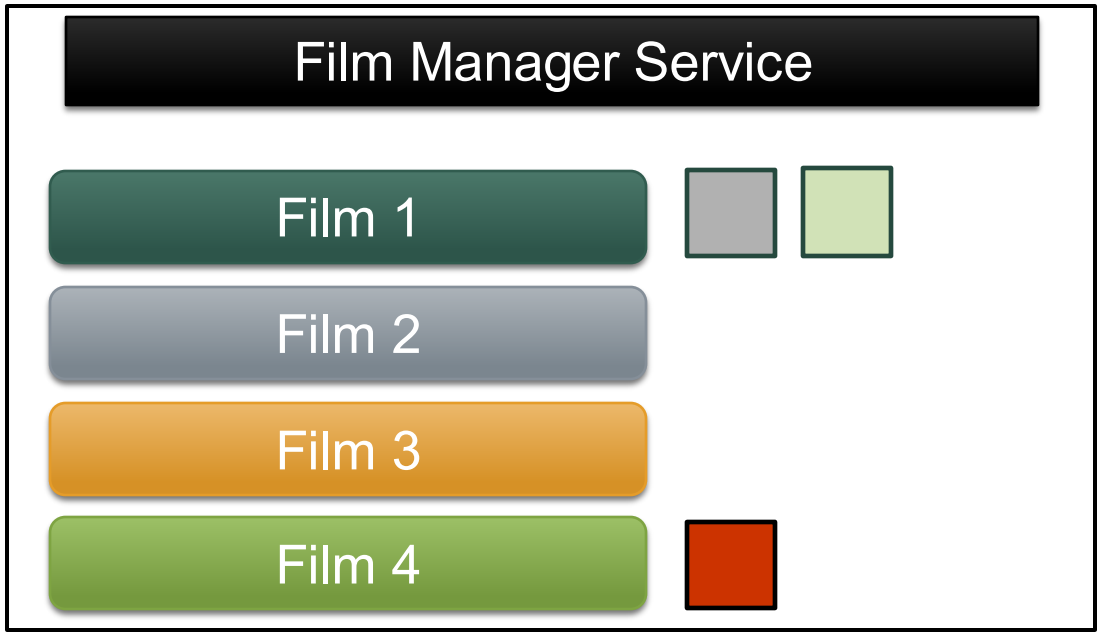


- A user can **delete** an image associated to a film (if she is the owner).
- The image is not saved anymore server-side.

Image Management in the Film Manager service (III)

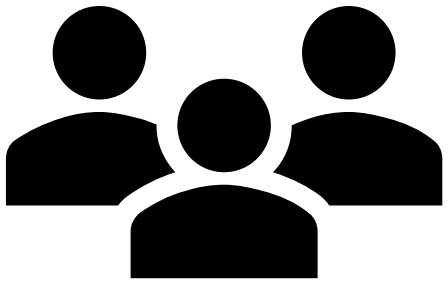






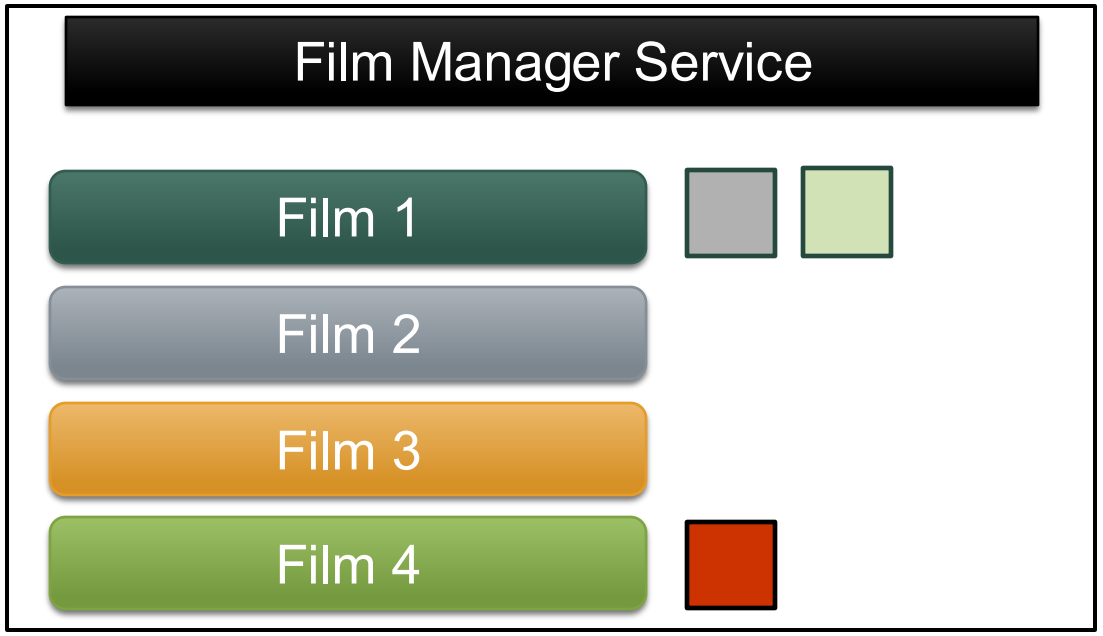
- A user can **retrieve all** images associated to a film (if she is the owner or a reviewer).
- The service will return a json-encoded array of *image* data structures, which do not contain the image files themselves.

Image Management in the Film Manager service (III)



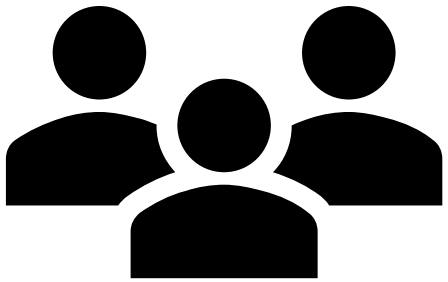
GET images
of Film 1





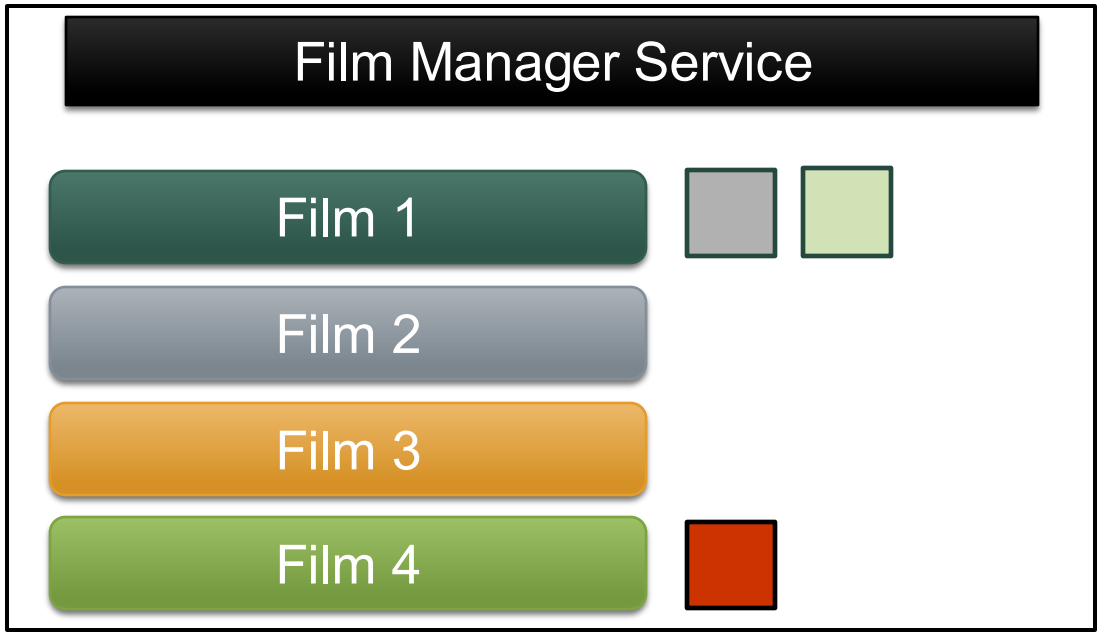
- A user can **retrieve all** images associated to a film (if she is the owner or a reviewer).
- The service will return a json-encoded array of *image* data structures, which do not contain the image files themselves.

Image Management in the Film Manager service (III)



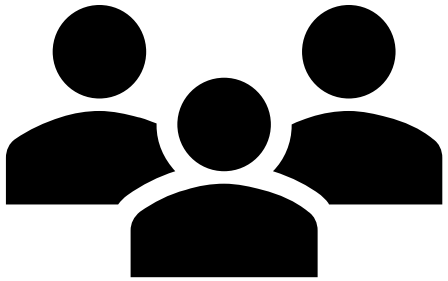
GET images
of Film 1

{…}, {…}

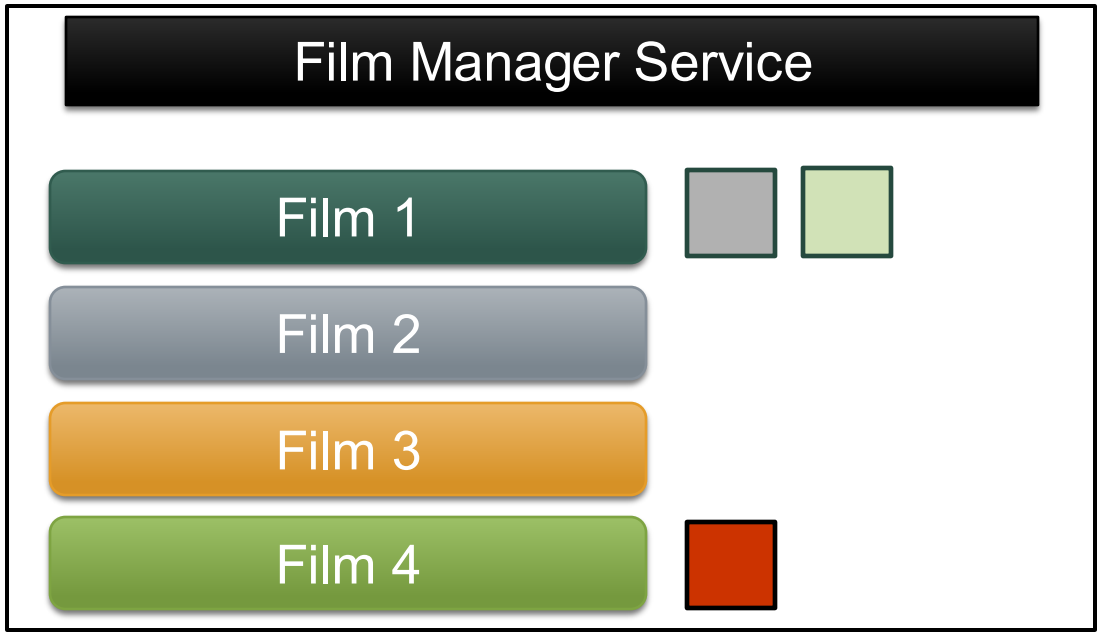


- A user can **retrieve all** images associated to a film (if she is the owner or a reviewer).
- The service will return a json-encoded array of *image* data structures, which do not contain the image files themselves.

Image Management in the Film Manager service (IV)

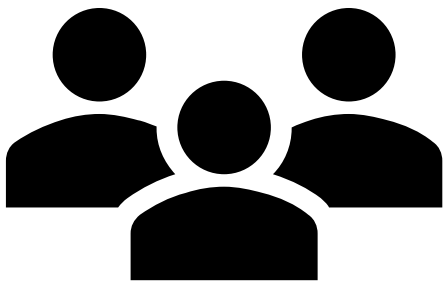




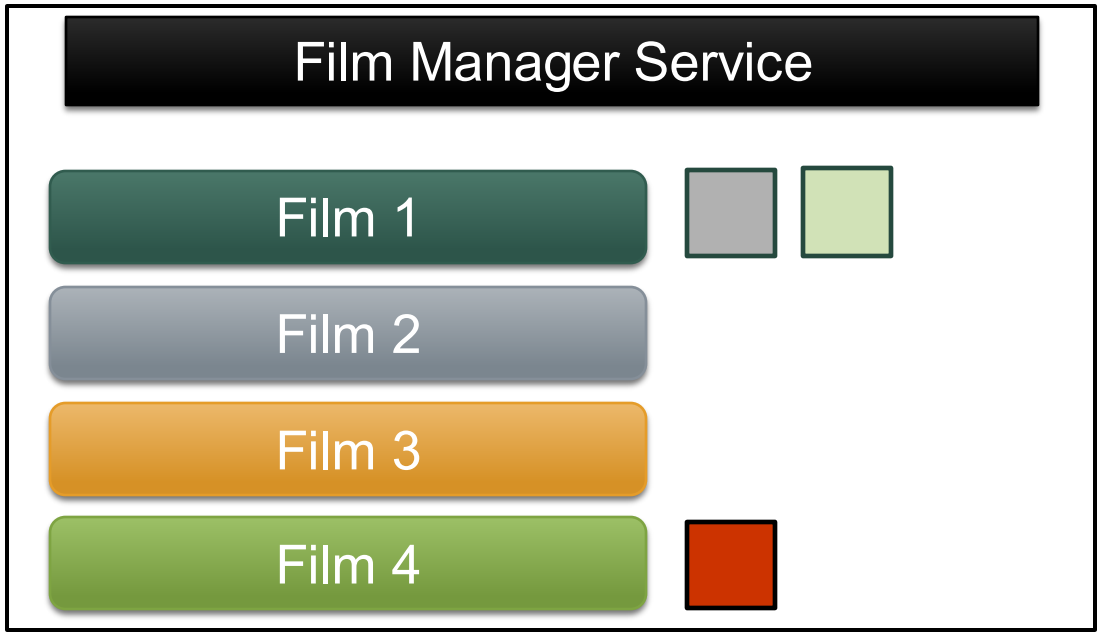


- A user can **retrieve** an image associated to a film (if she is the owner or a reviewer).
- The **Accept** header of the HTTP request specifies the content type.
- The user can decide whether to retrieve the image data structure (json content type), or the image file itself, in a supported image content type (image/png, image/jpg, and image/gif).
- In case the user requests another media type, the operation fails.

Image Management in the Film Manager service (IV)

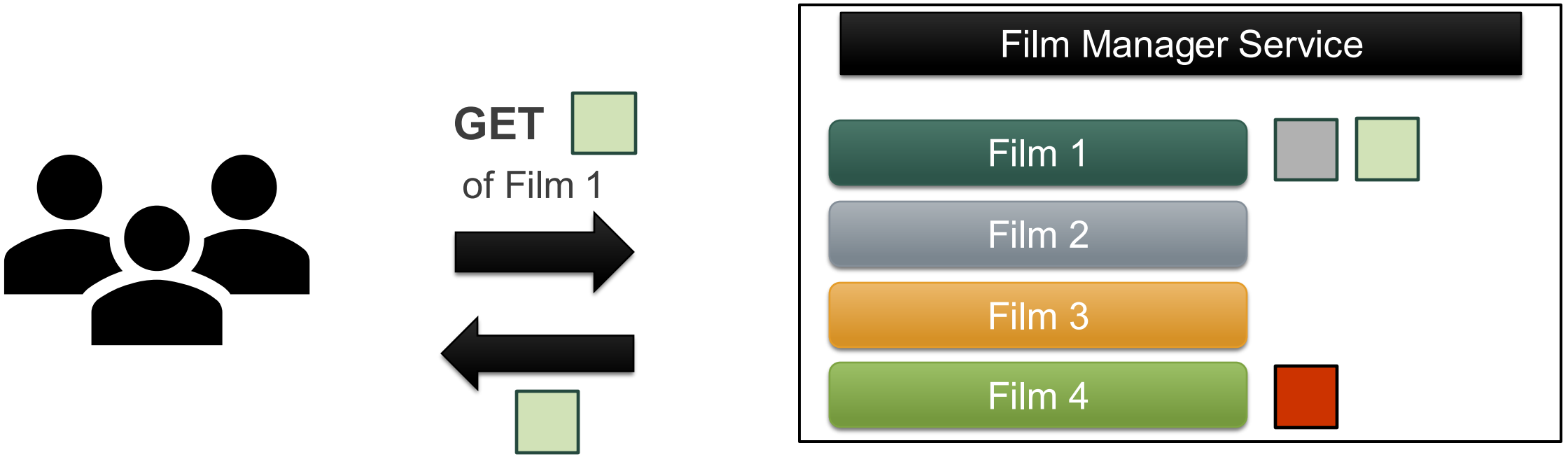






- A user can **retrieve** an image associated to a film (if she is the owner or a reviewer).
- The **Accept** header of the HTTP request specifies the content type.
- The user can decide whether to retrieve the image data structure (json content type), or the image file itself, in a supported image content type (image/png, image/jpg, and image/gif).
- In case the user requests another media type, the operation fails.

Image Management in the Film Manager service (IV)



- A user can **retrieve** an image associated to a film (if she is the owner or a reviewer).
- The **Accept** header of the HTTP request specifies the content type.
- The user can decide whether to retrieve the image data structure (json content type), or the image file itself, in a supported image content type (image/png, image/jpg, and image/gif).
- In case the user requests another media type, the operation fails.

Image Media Type Conversion

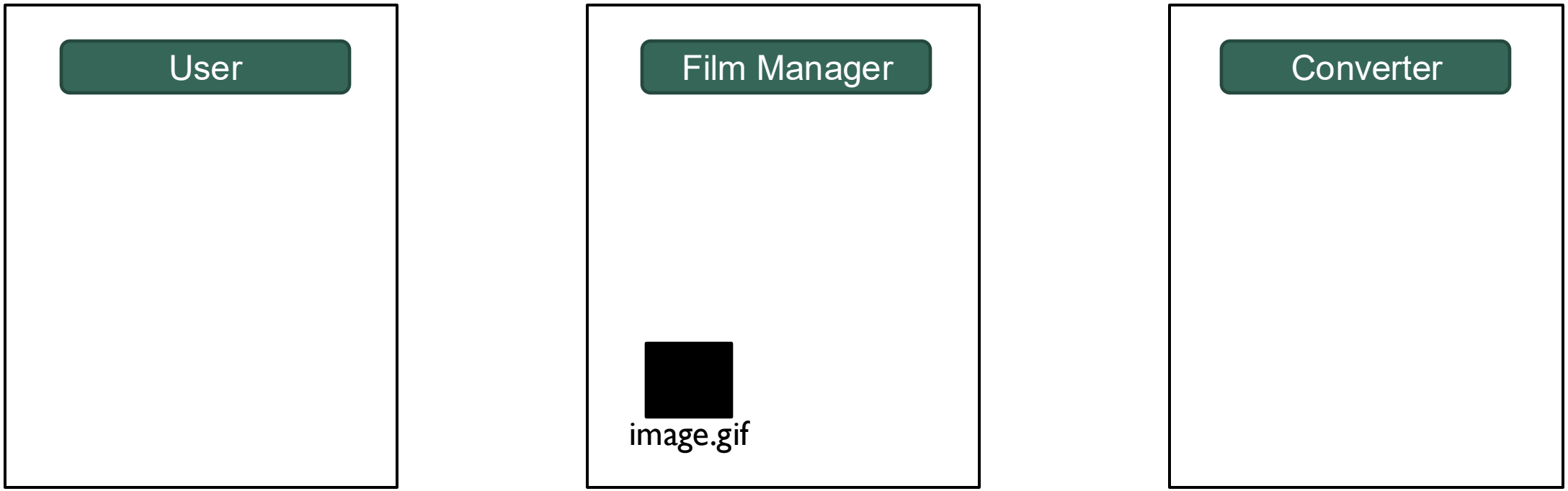


But… what happens if a requested image is saved in a **different media type** than the requested one?

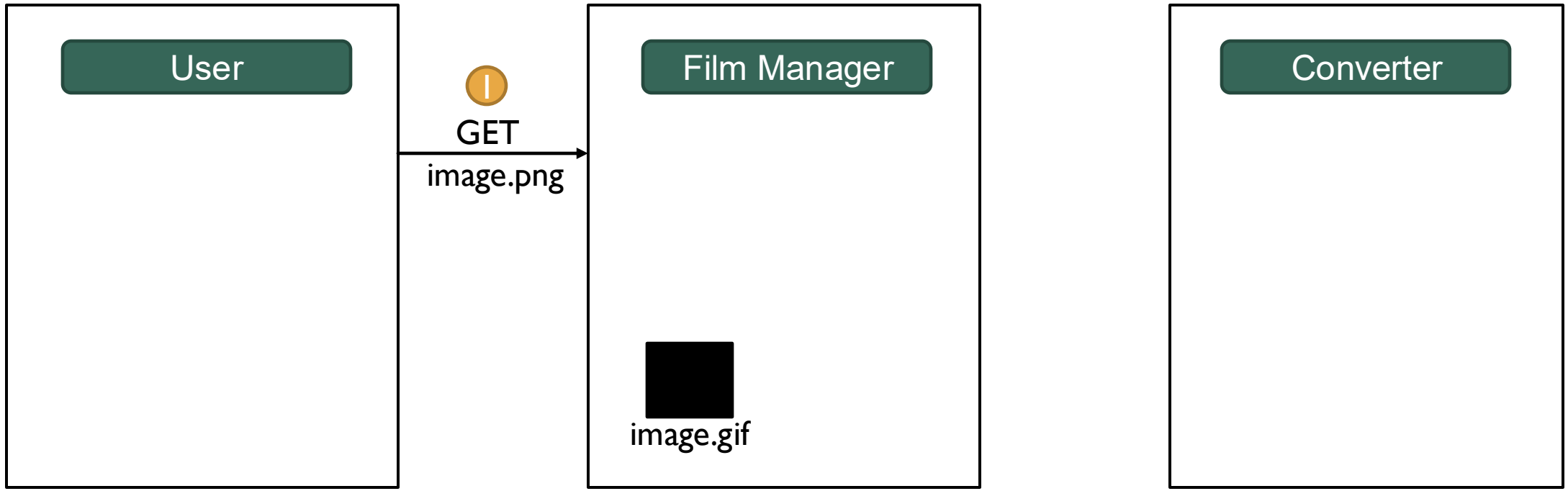
- The *Film Manager* service interacts with another service, called *Converter* service, **delegating** the operation of media type conversion.

- The *Film Manager* service creates a **gRPC channel** towards the *Converter* service (i.e., the *Film Manager* service covers the role of gRPC client, the *Converter* service covers the role of gRPC server).

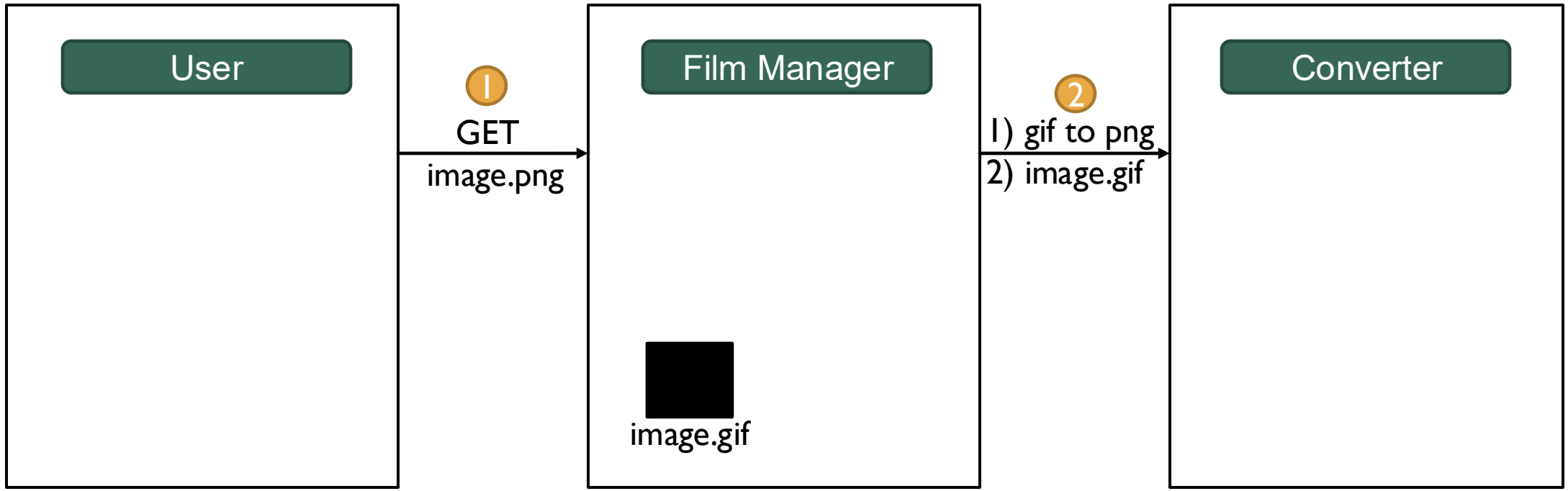
gRPC Communication



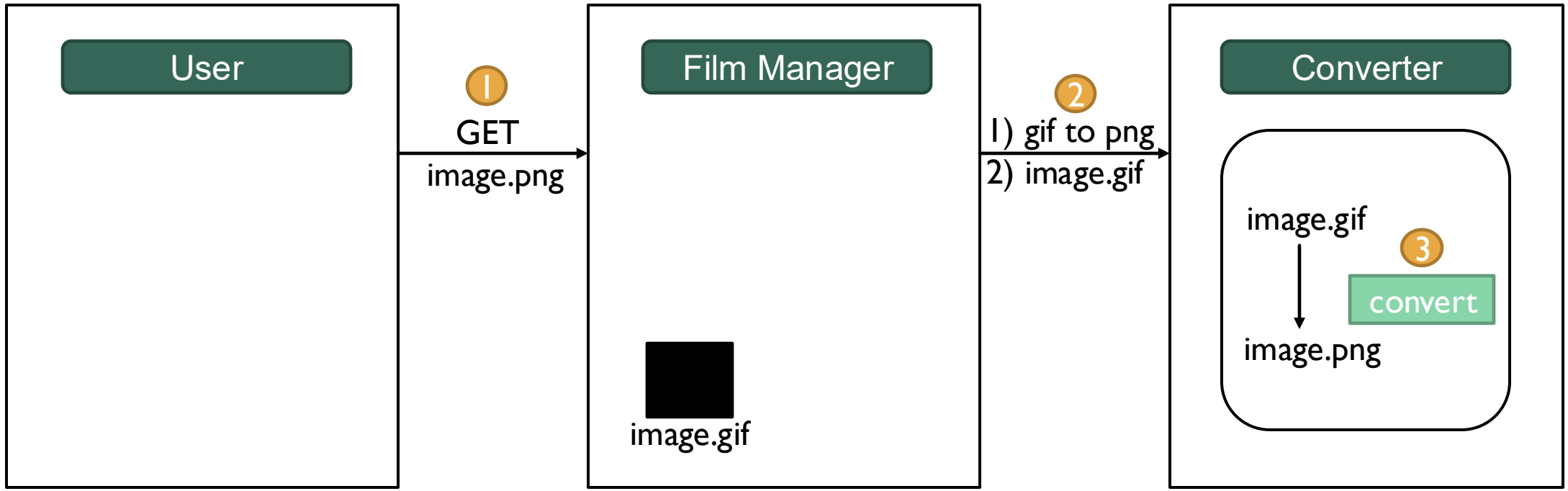
gRPC Communication



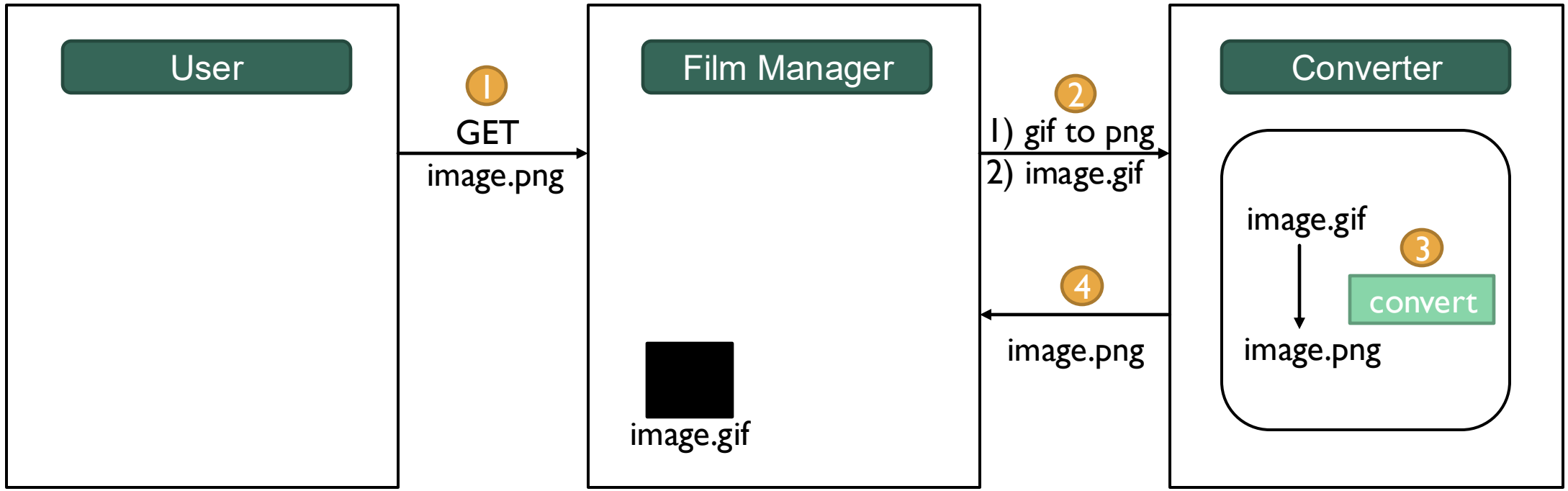
gRPC Communication



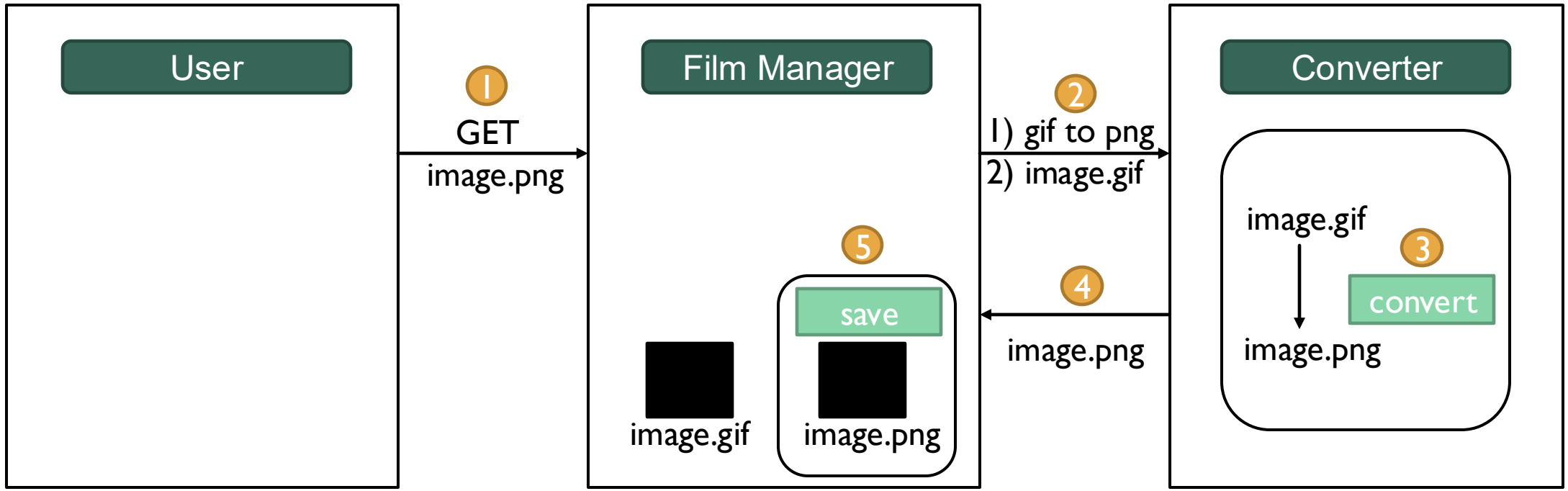
gRPC Communication



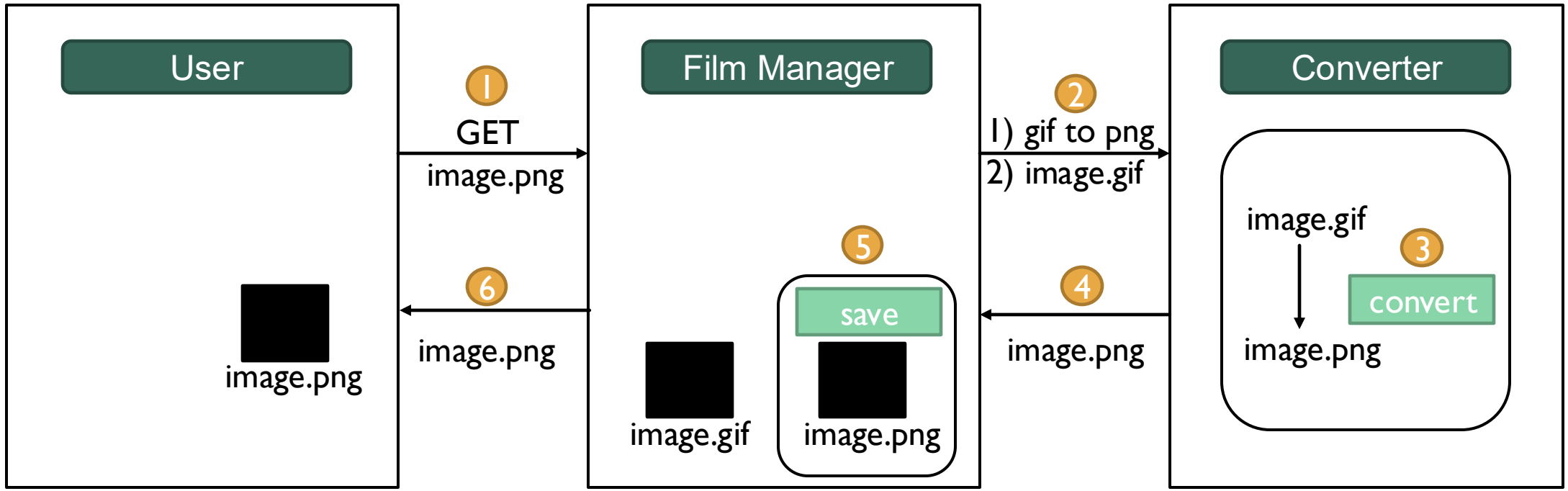
gRPC Communication



gRPC Communication



gRPC Communication



How to manage image files?



Image files must be managed in:

- **Postman**
  - send/receive images in HTTP requests/responses
- *Film Manager* service (**Node.js**)
  - send/receive images to/from Postman and the *Converter* service
  - locally save images
- *Converter* service (**Java**)
  - send/receive images to/from the *Film Manager* service
  - convert media types

How to manage image files?



Image files must be managed in:

- **Postman**
  - send/receive images in HTTP requests/responses
- *Film Manager* service (**Node.js**)
  - send/receive images to/from Postman and the *Converter* service
  - locally save images
- *Converter* service (**Java**)
  - send/receive images to/from the *Film Manager* service
  - convert media types



Let's see some tips for the image management!



Locally saving files with multer (I)



- Multer: node.js library to manage image storage (https://www.npmjs.com/package/multer)

components/storage.js

```
'use strict';
const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: function (req, file, cb) { cb(null, './uploads'); },
  filename: function (req, file, cb) {
    cb(null, Date.now() + '-' + Math.round(Math.random() * 1E9)
      + path.extname(file.originalname).toLowerCase());
  }
});

module.exports = storage;
```

Locally saving files with multer (II)



- In index.js, pass the storage to the fileUploader option of the OpenAPI validator middleware, which uses multer to handle multipart/form-data requests:

```
app.use(OpenApiValidator.middleware({
  apiSpec: ..., operationHandlers: ..., validateSecurity: ...,
  fileUploader: { storage: require('./components/storage.js') },
}));
```

- In the controller, the uploaded files can be accessed via the req.files property:

```
[ { fieldname: 'image', originalname: 'logo.png',
    encoding: '7bit', mimetype: 'image/png',
    destination: './uploads', filename: '1761040000000-482913574.png',
    path: 'uploads/1761040000000-482913574.png', size: 1785059 } ]
```

What you have to do



In this laboratory, you have to work on:

- **REST APIs**
  - define the image APIs in *api/openapi.json*, then run *npm run generate*
  - extend the database with the tables for the images
- *Film Manager* service (**Node.js**)
  - components: the image data structure (*image.js*) and the multer storage (*storage.js*)
  - controllers and services for the image management
  - gRPC client towards the *Converter* service (*@grpc/grpc-js*, *@grpc/proto-loader*)
- *Converter* service (**Java**)
  - already provided: study it, you may be asked to extend it in the exam



Thanks for your attention!

If you have further questions, use the
Slack Workspace or send an email

Rosario Rizza

rosario.rizza@polito.it



netgroup